

TECH

BROWSER APIs / DOM

BROWSER APIs / DOM

BROWSER APIs / DOM

30 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК developer.mozilla.org

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	OBSERVERS MUTATION Q1 · Q2 · Q3 · Q4 · Q5	INTERSECTION	RESIZE	Три наблюдателя — основа реактивного DOM, плюс правильный cleanup.
ДЕНЬ 2 СРЕДНИЙ	EVENT DELEGATION Q6 · Q7 · Q8 · Q9 · Q10	PASSIVE LISTENERS		Меньше слушателей, плавный скролл, аккуратное preventDefault.
ДЕНЬ 3 СРЕДНИЙ	CAPTURE / BUBBLE preventDefault Q11 · Q12 · Q13	stopPropagation		Каркас распространения событий — без него вся delegation непонятна.
ДЕНЬ 4 ТЯЖЁЛЫЙ	SHADOW DOM EVENTS Q14 · Q15 · Q16 · Q17 · Q18	FOCUS MANAGEMENT		Composed-events, фокус и ловушки в all Y.
ДЕНЬ 5 ТЯЖЁЛЫЙ	WEB WORKERS ERABLE Q19 · Q20 · Q21 · Q22 · Q23	COMMUNICATION	TRANSF	Параллельные вычисления вне main thread.
ДЕНЬ 6 ТЯЖЁЛЫЙ	SERVICE WORKER Q24 · Q25 · Q26 · Q27	CACHE API	OFFLINE	Сетевой прокси, кэширование, оффлайн-поведение.
ДЕНЬ 7 СРЕДНИЙ	PWA Q28 · Q29 · Q30	UPDATES	DEBUG SW	Progressive Web App: дизайн, обновления, отладка.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1

НАГРУЗКА

СРЕДНИЙ

5 ВОПРОСОВ

OBSERVERS**INTERSECTION****RESIZE****MUTATION****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Три наблюдателя – основа реактивного DOM, плюс правильный cleanup.

ВОПРОСЫ ДНЯ

- Q01 Когда использовать IntersectionObserver?
- Q02 Когда использовать ResizeObserver?
- Q03 Когда использовать MutationObserver?
- Q04 Какие риски у observer APIs?
- Q05 Как правильно чистить observers?

Когда использовать IntersectionObserver?

ОПИСАНИЕ

IntersectionObserver наблюдает за пересечением элемента с viewport (или с другим элементом-корнем) и асинхронно уведомляет, когда видимая часть меняется. Главное применение – lazy-load изображений и блоков, infinite scroll, sticky-эффекты, аналитика «сколько пользователей увидели этот блок». Дешевле, чем scroll-handler, потому что не блокирует main thread.

КАК РАБОТАЕТ

- 01 new IntersectionObserver(callback, options) – создаёт.
- 02 observer.observe(element) – начать наблюдать.
- 03 callback получает entries: каждый – IntersectionObserverEntry с isIntersecting, intersectionRatio, target.
- 04 options: root (по умолчанию viewport), rootMargin, threshold (массив порогов 0..1).
- 05 Asynchronous – браузер сам решает, когда вызывать.
- 06 Cleanup: observer.disconnect() или observer.unobserve(el).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Lazy-load картинок и тяжёлых компонентов.
- 02 Infinite scroll – наблюдаем за «sentinel» внизу списка.
- 03 Аналитика просмотров элементов.
- 04 Sticky / fade-in эффекты.
- 05 Оптимизация анимаций – паузить вне viewport.

ПОДВОДНЫЕ КАМНИ

- 01 Threshold 0 vs 1 – разный момент срабатывания.
- 02 Observer создаётся в render – каждый раз новый, утечка.
- 03 Cleanup в useEffect обязателен.
- 04 rootMargin не учитывает CSS-трансформации.
- 05 В iframe требует разрешения на cross-origin.

КОД

// Lazy-load изображения

```
useEffect(() => {
  const obs = new IntersectionObserver((entries) => {
    entries.forEach((e) => {
      if (e.isIntersecting) {
        e.target.src = e.target.dataset.src!;
      }
    });
  });
```

```
        obs.unobserve(e.target);
    }
    });
});
document.querySelectorAll('img[data-src]').forEach((el) => obs.observe(el));
return () => obs.disconnect();
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем IntersectionObserver лучше scroll-handler?
- 02 Что такое threshold-массив?
- 03 Как реализовать infinite scroll через IO?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API

Когда использовать ResizeObserver?

ОПИСАНИЕ

ResizeObserver уведомляет об изменении размеров элемента. Это гораздо лучше, чем слушать window.resize: реагирует на ЛЮБОЕ изменение размера элемента (не только viewport), не зависит от ре-флоу всей страницы. Применяется для адаптивных компонентов (container queries-like поведение), responsive charts, реакции на изменение sidebar.

КАК РАБОТАЕТ

```
01 new ResizeObserver(callback).observe(el).
02 Callback получает entries – каждый с contentRect / borderBoxSize /
contentBoxSize.
03 Box options: 'content-box' (default), 'border-box', 'device-pixel-content-box'.
04 Срабатывает асинхронно – после layout.
05 Cleanup: disconnect / unobserve.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Container-queries-like adaptive components.
02 Responsive charts (Chart.js / D3).
03 Виртуализация – пересчёт высоты строк.
04 Editor canvas / WYSIWYG.
05 Layout-adjustments при collapse/expand sidebar.
```

ПОДВОДНЫЕ КАМНИ

```
01 Цикл setState → resize → setState – infinite loop.
02 Срабатывает асинхронно – нельзя полагаться на sync-измерение.
03 Не реагирует на смену visibility/display:none → block.
04 В Safari старых версий некоторые поля отсутствуют.
```

КОД

// Адаптивный компонент

```
useEffect(() => {
  const obs = new ResizeObserver((entries) => {
    const w = entries[0].contentRect.width;
    setNarrow(w < 600);
  });
  obs.observe(ref.current!);
  return () => obs.disconnect();
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем ResizeObserver лучше window.resize?
- 02 Какие boxes доступны в callback?
- 03 Что такое container queries в CSS?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/ResizeObserver>

Когда использовать MutationObserver?

ОПИСАНИЕ

MutationObserver уведомляет об изменениях DOM-дерева: добавление/удаление узлов, изменения атрибутов, текстового содержимого. Используется в редких сценариях: интеграция с чужим кодом, который меняет DOM (тулзы, расширения), реакция на изменения third-party widget, тестовые helpers (waitFor элемента).

КАК РАБОТАЕТ

- 01 new MutationObserver(callback).observe(node, options).
- 02 options: childList, attributes, characterData, subtree, attributeOldValue, characterDataOldValue, attributeFilter.
- 03 Микрозадача: callback срабатывает асинхронно.
- 04 records – массив MutationRecord с type / addedNodes / removedNodes / oldValue.
- 05 takeRecords() – забрать накопленные изменения вручную.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Интеграция с DOM-меняющими библиотеками.
- 02 Браузерные расширения / userscripts.
- 03 Тестовые тулы (testing-library waitFor).
- 04 Полифиллы для отсутствующих API.
- 05 Логирование изменений DOM в дебаг-режиме.

ПОДВОДНЫЕ КАМНИ

- 01 Производительность: подписка на subtree большого дерева = много вызовов.
- 02 Плохо для частых изменений – overhead больше пользы.
- 03 Цикл: callback меняет DOM → новые мутации → callback снова.
- 04 Не реагирует на изменения CSS-стилей (для этого – ResizeObserver).

КОД

// Базовый пример

```
const obs = new MutationObserver((records) => {
  records.forEach((r) => {
    if (r.type === 'childList') {
      console.log('добавили:', r.addedNodes);
    }
  });
});
obs.observe(document.body, { childList: true, subtree: true });
```


МОГУТ ДОСПРОСИТЬ

- 01 Какие три типа мутаций бывают?
- 02 Что делает takeRecords?
- 03 Почему MutationObserver редко нужен в обычной разработке?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/MutationObserver>

Какие риски у observer APIs?

ОПИСАНИЕ

Главные риски: утечки памяти (observer без cleanup удерживает узлы и колбэки), бесконечные циклы (callback меняет состояние, что снова триггерит наблюдение), производительность (избыточные подписки, особенно у MutationObserver), асинхронность (нельзя полагаться на немедленный отклик).

КАК РАБОТАЕТ

- 01 Observer держит ссылку на node + callback. Без disconnect – утечка.
- 02 ResizeObserver-callback меняет размер → новый callback → цикл.
- 03 MutationObserver на subtree большого DOM – overhead.
- 04 Все колбэки async – race condition с другим кодом.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая работа с observers требует cleanup.
- 02 Code review – проверка на disconnect/unobserve.
- 03 Профилирование – observers в долго-живущих компонентах.

ПОДВОДНЫЕ КАМНИ

- 01 Observer вне useEffect – пересоздаётся на каждом рендере.
- 02 Cleanup не в return useEffect – утечка.
- 03 Подписка на document.body subtree без attributeFilter – performance kill.
- 04 Loop ResizeObserver: предотвращается через requestAnimationFrame или флаг.

КОД

// Защита от цикла

```
let isUpdating = false;
const obs = new ResizeObserver(() => {
  if (isUpdating) return;
  isUpdating = true;
  requestAnimationFrame(() => {
    update();
    isUpdating = false;
  });
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Как защититься от ResizeObserver-loop?
- 02 Когда disconnect, когда unobserve?

03 Что произойдёт без cleanup?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/ResizeObserver>

Как правильно чистить observers?

ОПИСАНИЕ

Любой observer должен быть остановлен, когда он больше не нужен. Способы:
observer.disconnect() – снимает ВСЕ подписки и освобождает callback;
observer.unobserve(element) – снимает только конкретный элемент. В React – внутри cleanup-функции useEffect. Без этого – утечки памяти и лишние вызовы.

КАК РАБОТАЕТ

- 01 disconnect() – полная остановка, observer можно переиспользовать после нового observe.
- 02 unobserve(el) – снять с одного элемента, остальные продолжают.
- 03 В React: useEffect → return () => obs.disconnect().
- 04 При unmount компонента cleanup гарантированно запустится.
- 05 В Vanilla JS – связать с lifecycle (например, removeChild → unobserve).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 ВСЕГДА для каждого observer в long-living коде.
- 02 Особенно в SPA с частой сменой страниц.
- 03 При работе с большими списками динамических элементов.

ПОДВОДНЫЕ КАМНИ

- 01 Создание observer вне useEffect – пересоздаётся, старый болтается.
- 02 Cleanup без disconnect – утечка.
- 03 Для одного элемента – unobserve экономнее, чем disconnect+observe заново.

КОД

// Шаблон cleanup

```
useEffect(() => {
  const obs = new IntersectionObserver(callback);
  obs.observe(ref.current!);
  return () => obs.disconnect();
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем disconnect отличается от unobserve?
- 02 Что произойдёт, если забыть cleanup в React?
- 03 Можно ли переиспользовать observer после disconnect?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/IntersectionObserver/disconnect>

EVENT DELEGATION**PASSIVE LISTENERS****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Меньше слушателей, плавный скролл, аккуратное `preventDefault`.

ВОПРОСЫ ДНЯ

- Q06 Что такое event delegation?
- Q07 Когда event delegation полезен?
- Q08 Что такое passive event listener?
- Q09 Почему passive listeners важны для scroll?
- Q10 Когда passive listener нельзя использовать?

ВОПРОС Q06

Что такое event delegation?

ОПИСАНИЕ

Event delegation – паттерн, при котором обработчик вешается на общего родителя, а не на каждый элемент-ребёнок. Обработчик использует `event.target`, чтобы понять, на каком конкретно элементе случился клик. Возможно благодаря bubbling: события поднимаются от `target` до `document`. Меньше обработчиков – меньше памяти и быстрее работа с динамическими списками.

КАК РАБОТАЕТ

- 01 Один listener на родителе – реагирует на все события детей.
- 02 `event.target` – конкретный элемент, на котором произошло событие.
- 03 `event.currentTarget` – элемент, на котором висит listener.
- 04 `closest(selector)` внутри обработчика – найти ближайший матч.
- 05 Динамически добавленные дети тоже работают без новой регистрации.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Длинные списки с одинаковым поведением (`todo`, `messages`).
- 02 Динамически добавляемые элементы.
- 03 Таблицы с `inline-actions` (кнопки в каждой строке).
- 04 Меню с выпадающими элементами.

ПОДВОДНЫЕ КАМНИ

- 01 `event.target` может быть вложенным элементом (icon внутри button) – нужен `closest`.
- 02 Не работает для `focus/blur` (они не bubble) – есть `focusin/focusout`.
- 03 `stopPropagation` внутри ребёнка ломает delegation.
- 04 В Shadow DOM events изолированы (см. Q15).

КОД

// Базовый пример

```
list.addEventListener('click', (e) => {
  const li = (e.target as HTMLElement).closest('li');
  if (!li) return;
  doAction(li.dataset.id!);
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `event.target` отличается от `currentTarget`?
- 02 Какие события не всплывают?
- 03 Зачем `closest` в обработчике?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Events#event_delegation

ВОПРОС Q07

Когда event delegation полезен?

ОПИСАНИЕ

Полезен, когда: много однотипных элементов с одинаковым поведением (списки, ячейки таблицы); элементы создаются/удаляются динамически (не нужно перевешивать обработчики); хочется уменьшить количество слушателей (memory + performance). Не нужен для уникальных кнопок и для событий, которые не bubble (focus, blur, scroll, resize).

КАК РАБОТАЕТ

- 01 1 listener < 1000 listeners – экономия памяти.
- 02 Динамика без перевешивания – упрощение кода.
- 03 Меньше работ для GC при unmount.
- 04 Современный React уже использует root-level delegation под капотом.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Списки сообщений / комментариев / товаров.
- 02 Таблицы с инлайн-действиями.
- 03 Меню, breadcrumbs, табы.
- 04 Tree-views, drag-drop по дереву.

ПОДВОДНЫЕ КАМНИ

- 01 Для нативных событий формы (input, change) обычно удобнее прямое подключение.
- 02 Если поведение разное для каждого ребёнка – delegation усложняет код.
- 03 Сложные closest-селекторы становятся хрупкими.

КОД

// В React (16+) внутренний механизм

```
// React делает root-delegation сам – все onClick  
// попадают в один listener на root, потом // раздаются.  
// Это причина, почему addEventListener в обход // React может не сработать как ожидае  
сь.
```

МОГУТ ДОСПРОСИТЬ

- 01 Как делает React event delegation внутри?
- 02 Когда лучше прямое подключение?
- 03 Что произойдёт при stopPropagation в дочернем?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Events

Что такое passive event listener?

ОПИСАНИЕ

Passive listener – обработчик с опцией `passive: true`, который ОБЕЩАЕТ браузеру не вызывать `preventDefault`. Браузер благодаря этому может не ждать обработчика перед скроллом – тач-скролл становится плавным. Применяется к `touchstart/touchmove/wheel` – событиям, которые иначе блокируют рендеринг.

КАК РАБОТАЕТ

- 01 `addEventListener('touchmove', cb, { passive: true })`.
- 02 Внутри `cb` `preventDefault` не работает (warning в консоли).
- 03 Браузер начинает обработку `scroll/zoom` параллельно с обработчиком.
- 04 Современные браузеры по умолчанию `passive=true` для `touchstart/touchmove/wheel` на `window/document`.
- 05 `passive=false` – явно нужно блокировать default (например, кастомный `pinch-to-zoom`).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой обработчик `scroll-related` событий.
- 02 Touch-обработчики на `mobile`.
- 03 `Wheel handlers` для дашбордов.
- 04 Аналитика по `scroll` (без `preventDefault`).

ПОДВОДНЫЕ КАМНИ

- 01 `preventDefault` не сработает в `passive` – нужно явно `passive: false` если нужно блокировать.
- 02 По умолчанию `passive=true` для некоторых событий – сюрприз, если не знать.
- 03 `Eslint/lighthouse` – флагают `non-passive listeners` как warning.

КОД

```
// Passive scroll handler
window.addEventListener('scroll', onScroll, { passive: true });
// onScroll не может вызвать preventDefault

// Когда нужен preventDefault
el.addEventListener('touchstart', onStart, { passive: false });
// внутри можно e.preventDefault()
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие события `passive=true` по умолчанию?
- 02 Что покажет `lighthouse`?

03 Чем passive отличается от capture?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener#using_passive_listeners

Почему passive listeners важны для scroll?

ОПИСАНИЕ

Без passive браузер обязан подождать обработчик, чтобы узнать — будет ли preventDefault (отмена скrolла). Это блокирует main thread на десятки миллисекунд: scroll "подвисает". С passive=true браузер сразу скроллит, а обработчик выполняется параллельно. Это критично для мобильных устройств и тачскринов.

КАК РАБОТАЕТ

- 01 Браузер прокладывает компромисс: если обработчик может отменить — нужно ждать.
- 02 Passive — обещание не отменять. Браузер делает scroll сразу.
- 03 Latency reduction: 100-500мс на slow devices.
- 04 Lighthouse score: «Does not use passive listeners» = warning.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Mobile-веб приложения с touch-навигацией.
- 02 Long-page прокрутка.
- 03 Lazy-loaded ленты.
- 04 Animation-on-scroll сценарии.

ПОДВОДНЫЕ КАМНИ

- 01 Если случайно вызовете preventDefault внутри passive — warning, эффекта нет.
- 02 Сторонние библиотеки могут вешать non-passive — проверять.
- 03 В IE / старых Safari опция passive не понимается — feature-detect.

КОД

// Чек-лист

```
// scroll: passive по умолчанию в современных браузерах
// touchmove на el — задаём явно
el.addEventListener('touchmove', onMove, { passive: true });
```

МОГУТ ДОСПРОСИТЬ

- 01 Какой эффект на FID/INP?
- 02 Что флагает Lighthouse?
- 03 Как проверить passive поддерживается?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/uses-passive-event-listeners>

ВОПРОС Q10

Когда `passive listener` нельзя использовать?

ОПИСАНИЕ

Когда обработчику ОБЯЗАТЕЛЬНО нужно вызвать `preventDefault`: кастомные жесты (`pinch-to-zoom`, `swipe-to-delete`, `drag-and-drop` с нативными обработчиками), запрет скролла внутри модалки, блокировка `long-press` на конкретных элементах. В таких случаях явно ставится `passive: false`.

КАК РАБОТАЕТ

- 01 `passive: false` → `preventDefault` работает, скролл ждёт.
- 02 Используется точно, не на всё приложение.
- 03 Часто комбинируется с `touch-action: none` в CSS – альтернатива через CSS.
- 04 Если можно решить через `touch-action` – лучше так (меньше JS).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Custom gesture-handlers (`pinch`, `swipe`).
- 02 Drag-drop элементов внутри `scroll`-контейнера.
- 03 Модалки, блокирующие `background-scroll`.
- 04 Игровые UI с `tap-to-action`.

ПОДВОДНЫЕ КАМНИ

- 01 Глобальный `passive: false` – kill mobile UX.
- 02 `preventDefault` внутри `pull-to-refresh` запрещает поведение браузера.
- 03 Лучше использовать CSS `touch-action`, где возможно.
- 04 Старые библиотеки могут не уважать `passive` – проверять.

КОД

// Точно

```
swipeArea.addEventListener('touchmove', onSwipe, { passive: false });  
// внутри onSwipe можно e.preventDefault()
```

// Через CSS – без JS

```
.no-pan { touch-action: none; }  
/* отключает scroll/zoom для элемента */
```

МОГУТ ДОСПРОСИТЬ

- 01 Что делает `touch-action: none`?
- 02 Какие альтернативы `preventDefault`?
- 03 Что лучше – JS-`passive=false` или CSS?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/CSS/touch-action>

ДЕНЬ 3 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

CAPTURE / BUBBLE stopPropagation preventDefault**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Каркас распространения событий — без него вся delegation непонятна.

ВОПРОСЫ ДНЯ

- Q11 Как работает capturing и bubbling?
- Q12 Что делает stopPropagation?
- Q13 Что делает preventDefault?

Как работает capturing и bubbling?

ОПИСАНИЕ

DOM-события распространяются в три фазы: capturing (сверху от window к target), target (на самом target), bubbling (снизу обратно к window). По умолчанию обработчики срабатывают на bubbling-фазе. Capturing включается через `addEventListener('event', cb, { capture: true })`. Не все события bubble (focus, blur, scroll на element, mouseenter).

КАК РАБОТАЕТ

- 01 Phase 1 (CAPTURING_PHASE = 1): window → target.
- 02 Phase 2 (AT_TARGET = 2): на самом target.
- 03 Phase 3 (BUBBLING_PHASE = 3): target → window.
- 04 `addEventListener(name, cb, true|{capture:true})` – ловить в capture.
- 05 По умолчанию – bubbling.
- 06 `event.eventPhase` – текущая фаза.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Capturing – для перехвата ДО того, как событие дойдёт до target (например, на root для аналитики).
- 02 Bubbling – основа event delegation.
- 03 `stopPropagation` – остановить дальнейшее распространение.

ПОДВОДНЫЕ КАМНИ

- 01 Часть событий не bubble (focus → focusin для bubble).
- 02 scroll на конкретном элементе не bubble; на window – bubble.
- 03 `stopPropagation` в capture останавливает bubble тоже.
- 04 В React события synthetic – bubble через React-tree, не через DOM.

КОД

// Capture vs bubble

```
outer.addEventListener('click', () => log('outer bubble'));
outer.addEventListener('click', () => log('outer capture'), true);
inner.addEventListener('click', () => log('inner'));
// клик на inner → outer capture, inner, outer bubble
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие события не bubble?
- 02 Что такое focusin/focusout?
- 03 Чем `event.target` от `currentTarget` отличается?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Event_bubbling

Что делает stopPropagation?

ОПИСАНИЕ

`event.stopPropagation()` прекращает дальнейшее распространение события — остальные обработчики на родителях не сработают. `stopImmediatePropagation()` — ещё и не вызовет другие обработчики на ТОМ ЖЕ элементе. Используется когда поведение должно быть строго локальным; но злоупотребление ломает event delegation и аналитику на root.

КАК РАБОТАЕТ

- 01 `stopPropagation` — остановить bubble/capture дальше.
- 02 `stopImmediatePropagation` — то же + остальные handlers на текущем элементе.
- 03 Не отменяет default-поведение (для этого `preventDefault`).
- 04 В React synthetic-событие тоже имеет эти методы — работает в React-tree.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Модалки — клик внутри не должен закрывать диалог (`stopPropagation` на `overlay-listener`).
- 02 Custom-context-menu — стоп нативного.
- 03 Изоляция UI-компонента в чужой странице.

ПОДВОДНЫЕ КАМНИ

- 01 Ломает event delegation — root-listener не сработает.
- 02 Ломает аналитику, висящую на window/document.
- 03 В React `stopPropagation` в synthetic не остановит native bubble (если повесили `addEventListener`).
- 04 `stopImmediatePropagation` удаляет «соседей» — иногда сюрприз.

КОД

// Модалка — клик внутри

```
modalContent.addEventListener('click', (e) => {
  e.stopPropagation();
});
overlay.addEventListener('click', () => closeModal());
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `stopPropagation` отличается от `preventDefault`?
- 02 Что такое `stopImmediatePropagation`?
- 03 Почему `React.stopPropagation` иногда не помогает?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/Event/stopPropagation>

Что делает preventDefault?

ОПИСАНИЕ

`event.preventDefault()` отменяет действие браузера по умолчанию – переход по ссылке, отправку формы, контекстное меню, drag-and-drop. НЕ останавливает распространение события (для этого `stopPropagation`). Часто используется в SPA для перехвата link-кликов и кастомной навигации.

КАК РАБОТАЕТ

- 01 Отменяет default browser action.
- 02 Не работает в passive listeners (см. Q8).
- 03 `event.defaultPrevented` – флаг, был ли вызов.
- 04 Не отменяет propagation.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Кастомная навигация по `<a href>`.
- 02 `form.onsubmit` – собственная отправка через AJAX.
- 03 Drag-drop – отменить default behaviour.
- 04 Custom context menu (на right-click).
- 05 Запрет copy/paste для secure-полей.

ПОДВОДНЫЕ КАМНИ

- 01 Не работает в passive event – warning.
- 02 Не останавливает распространение – используют пару с `stopPropagation`, если нужно.
- 03 Иногда отменяется уже не нужный default (контекст ушёл).
- 04 `Form.preventDefault` внутри submit – без него обычная отправка.

КОД

```
// Form-submit
form.addEventListener('submit', (e) => {
  e.preventDefault();
  fetch('/api', { method: 'POST', body: new FormData(form) });
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `preventDefault` отличается от `stopPropagation`?
 - 02 Когда работает в passive?
 - 03 Что такое `defaultPrevented`?
-

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/Event/preventDefault>

SHADOW DOM EVENTS**FOCUS MANAGEMENT****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Composed-events, фокус и ловушки в ally.

ВОПРОСЫ ДНЯ

- Q14 Что такое composed events?
- Q15 Как работают events в Shadow DOM?
- Q16 Как работает focus management?
- Q17 Как программно управлять focus?
- Q18 Как избежать focus trap bugs?

ВОПРОС Q14

Что такое composed events?

ОПИСАНИЕ

Composed event – событие, которое может «выйти» из Shadow DOM в обычное дерево. По умолчанию события внутри Shadow DOM не bubble наружу – они перенаправляются (re-targeted) и видны снаружи как будто шли от shadow-root host. Свойство composed: true позволяет событию пересечь границу. Большинство user-input событий (click, focus, keydown) – composed.

КАК РАБОТАЕТ

- 01 Внутри Shadow DOM event.target – реальный элемент.
- 02 Снаружи event.target – host (компонент).
- 03 event.composed – true/false.
- 04 event.composedPath() – массив всех элементов на пути, включая shadow.
- 05 Кастомные события – composed по умолчанию false.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Web Components – нужно уведомить родителя.
- 02 Кастомные events для коммуникации.
- 03 Аналитика, когда хочется видеть конкретный элемент.

ПОДВОДНЫЕ КАМНИ

- 01 Кастомный CustomEvent без composed:true не выйдет за shadow – сюрприз.
- 02 event.target вне shadow – не реальный элемент, а host.
- 03 composedPath() работает в shadow, но не в полностью изолированных контекстах.
- 04 Не все события composed – focus/blur не выходят.

КОД

```
// Custom composed event
```

```
this.dispatchEvent(new CustomEvent('change', {  
  bubbles: true,  
  composed: true,  
  detail: { value: 'x' },  
}));
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое composedPath?
- 02 Какие нативные события composed по умолчанию?
- 03 Почему focus не composed?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/Event/composed>

ВОПРОС Q15

Как работают events в Shadow DOM?

ОПИСАНИЕ

События в Shadow DOM ведут себя по особым правилам: внутри shadow-tree всё как обычно (capture, bubble, target). Но при пересечении границы shadow → light DOM event.target меняется на host (re-targeting), и событие выходит наружу только если composed: true. Это сделано для инкапсуляции – внешний код не видит внутренних деталей компонента.

КАК РАБОТАЕТ

- 01 Re-targeting: за пределами shadow target = host.
- 02 composed: true – событие пересекает границу.
- 03 composedPath() – полный путь, включая shadow nodes.
- 04 Слушатели внутри shadow видят оригинальный target.
- 05 Слововое содержимое (slotted) – slot имеет свой slotchange event.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Custom Elements / Web Components.
- 02 Lit / Stencil библиотеки.
- 03 Полная изоляция компонента (например, embed-виджеты).

ПОДВОДНЫЕ КАМНИ

- 01 event.target снаружи shadow ≠ реальный элемент – путает.
- 02 Аналитика по target ломается, если она вне shadow.
- 03 stopPropagation внутри shadow не остановит re-targeted event снаружи (хотя обычно так и нужно).
- 04 Сложно дебажить – event-flow не очевиден.

КОД

// Внутри/снаружи

```
// shadowRoot.querySelector('button')  
// click → внутри: target=button  
//          снаружи: target=my-component (host)
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое re-targeting?
- 02 Когда нужен composed: true?
- 03 Что такое slotchange?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_shadow_DOM

Как работает focus management?

ОПИСАНИЕ

Focus — текущий «активный» элемент, способный получать клавиатурный ввод. `document.activeElement` показывает кто в фокусе. Управление фокусом — ключевая часть accessibility: tab-навигация, screen readers, keyboard-only пользователи. В SPA и модалках разработчик ОБЯЗАН вручную управлять фокусом, иначе keyboard-навигация ломается.

КАК РАБОТАЕТ

- 01 `tabindex=0` — элемент в обычном tab-order.
- 02 `tabindex=-1` — фокусируется только программно.
- 03 `tabindex>0` — кастомный порядок (анти-паттерн).
- 04 `el.focus()` / `el.blur()` — программно.
- 05 FocusEvent: `focus`, `blur` (не `bubble`); `focusin`, `focusout` (`bubble`).
- 06 `document.activeElement` — кто сейчас в фокусе.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Открытие модалки — переместить фокус внутрь.
- 02 Заккрытие модалки — вернуть фокус на trigger.
- 03 Skip-links ("перейти к контенту").
- 04 Auto-focus first invalid field в форме.
- 05 Custom-комбобокс с keyboard navigation.

ПОДВОДНЫЕ КАМНИ

- 01 `Tabindex>0` ломает естественный порядок.
- 02 Auto-focus при загрузке страницы — раздражает users.
- 03 Без management фокус «уплывает» на body после удаления элемента.
- 04 Скрытые элементы (`display:none`) теряют фокус — не уведомляя пользователя.

КОД

// Управление

```
// фокус первого input в модалке
useEffect(() => {
  if (open) modalRef.current?.querySelector('input')?.focus();
}, [open]);
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие элементы фокусируются по умолчанию?

02 Что делает `tabindex=-1`?

03 Чем `focusin` отличается от `focus`?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement/focus>

Как программно управлять focus?

ОПИСАНИЕ

Через `el.focus()` и `el.blur()`. Дополнительно: `tabIndex` для управления порядком, `focus-visible` для стилизации только клавиатурного фокуса, `FocusOptions` для `preventScroll`. В библиотеках (`react-aria`, `focus-trap`) есть готовые трекеры фокуса для модальных и ровер'ов.

КАК РАБОТАЕТ

- 01 `el.focus({ preventScroll: true })` – фокус без скrolла к элементу.
- 02 `el.blur()` – снять фокус.
- 03 `tabIndex` как атрибут / property.
- 04 `:focus-visible` – CSS-pseudo для клавиатурного фокуса.
- 05 `react-aria` / `floating-ui` – готовые решения.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Модалки/диалоги.
- 02 Wizard'ы – фокус на следующем шаге.
- 03 Form-validation – фокус на первой ошибке.
- 04 Search – auto-focus input при открытии.
- 05 Ally – skip-links.

ПОДВОДНЫЕ КАМНИ

- 01 `focus()` на элемент в момент анимации – может не сработать.
- 02 `preventScroll` не везде поддерживается.
- 03 Auto-focus раздражает на mobile (открывает клавиатуру).
- 04 Без `focus-visible` – синяя рамка везде, что выглядит странно.

КОД

// Auto-focus в модалке

```
useEffect(() => {  
  if (open) firstInputRef.current?.focus({ preventScroll: true });  
}, [open]);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `focus-visible`?
- 02 Зачем `preventScroll`?
- 03 Чем `focus-trap` полезен?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement/focus>

Как избежать focus trap bugs?

ОПИСАНИЕ

Focus trap – удержание фокуса внутри ограниченной области (модалка, диалог), пока она открыта. Tab/Shift+Tab должны заикликоваться внутри. Антипаттерны: кнопки/ссылки вне трапа доступны Tab'ом; фокус «уезжает» при удалении элемента; после закрытия фокус не возвращается на trigger. Готовые решения – focus-trap, react-aria.

КАК РАБОТАЕТ

- 01 Найти первый/последний фокусируемый внутри контейнера.
- 02 Tab на последнем → фокус на первый. Shift+Tab на первом → на последний.
- 03 При открытии – запомнить активный элемент.
- 04 При закрытии – вернуть фокус.
- 05 Скрыть остальной UI от screen-reader через aria-hidden или inert.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Модалки.
- 02 Drawer'ы / sidebar'ы.
- 03 Comboboxes / dropdowns.
- 04 Tooltips на focus.

ПОДВОДНЫЕ КАМНИ

- 01 Не вернуть фокус на trigger – пользователь теряется.
- 02 Не скрыть остальную страницу – screen-reader путается.
- 03 Focusable элементы добавлены динамически – trap пропускает.
- 04 Стандартный <dialog> делает много из этого автоматически – не дублировать.

КОД

// Возврат фокуса

```
useEffect(() => {
  if (!open) return;
  const prev = document.activeElement as HTMLElement;
  modalRef.current?.querySelector<HTMLElement>('input,button')?.focus();
  return () => prev?.focus();
}, [open]);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что делает <dialog>?
- 02 Что такое inert?
- 03 Чем focus-trap отличается от ручной реализации?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/dialog>

WEB WORKERS**COMMUNICATION****TRANSFERABLE****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Параллельные вычисления вне main thread.

ВОПРОСЫ ДНЯ

- Q19 Что такое Web Worker?
- Q20 Когда использовать Web Worker?
- Q21 Что нельзя делать в Web Worker?
- Q22 Как происходит communication с Worker?
- Q23 Что такое transferable objects?

ВОПРОС Q19

Что такое Web Worker?

ОПИСАНИЕ

Web Worker – отдельный поток JS, в котором можно выполнять CPU-интенсивные вычисления, не блокируя main thread. Worker не имеет доступа к DOM и большинству Web API; общается с main через postMessage / onmessage. Используется для тяжёлой обработки данных, шифрования, парсинга больших файлов.

КАК РАБОТАЕТ

```
01 new Worker('worker.js') – создание.  
02 Внутри worker – собственный global (self), без window/document.  
03 Communication: worker.postMessage(data); worker.onmessage = e => ....  
04 ImportScripts() – загрузка дополнительных скриптов (в classic worker).  
05 Module worker (type: 'module') – поддерживает import.  
06 worker.terminate() – остановка.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Тяжёлые вычисления (image processing, hash, AI).  
02 Парсинг больших JSON/CSV.  
03 Crypto-операции.  
04 OffscreenCanvas рендер.  
05 Search-индексация на клиенте.
```

ПОДВОДНЫЕ КАМНИ

```
01 Нет доступа к DOM – все UI-обновления через main thread.  
02 Создание worker'a – overhead (~10ms).  
03 Большие данные через postMessage копируются – медленно (см. transferable).  
04 Не все библиотеки работают в worker.
```

КОД

// Базовое

```
// main.ts  
const worker = new Worker(new URL('./worker.ts', import.meta.url), { type: 'module' });  
worker.postMessage({ data: 'big' });  
worker.onmessage = (e) => console.log(e.data);  
  
// worker.ts  
self.onmessage = (e) => {  
  const result = heavyCompute(e.data);  
  self.postMessage(result);  
}
```

```
};
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие API недоступны в worker?
- 02 Что такое module worker?
- 03 Когда terminate?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers

ВОПРОС Q20

Когда использовать Web Worker?

ОПИСАНИЕ

Когда основной поток заблокирован тяжёлой задачей и это видно пользователю: лагает анимация, не отвечает ввод. Подходит для: шифрования/хеширования, обработки изображений, парсинга больших файлов, компиляции TypeScript на клиенте, поиска по большому индексу. Не подходит для DOM-операций и для маленьких задач (overhead больше выгоды).

КАК РАБОТАЕТ

- 01 Признак: long-task >50ms блокирует UI.
- 02 Worker pool – для нескольких параллельных задач.
- 03 Comlink – упрощённое RPC поверх postMessage.
- 04 OffscreenCanvas – рендеринг canvas в worker.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Crypto: bcrypt, hashing.
- 02 Image: resize, compress.
- 03 Парсинг: PDF, XLSX, JSON 100+MB.
- 04 Search: full-text index.
- 05 Real-time analytics.

ПОДВОДНЫЕ КАМНИ

- 01 Overhead ~10ms на создание + serialize данных.
- 02 Маленькие задачи дешевле сделать sync.
- 03 Сложность дебага – другая среда.
- 04 Кросс-браузерные нюансы Module Workers.

КОД

```
// Comlink

// worker.ts
import * as Comlink from 'comlink';
Comlink.expose({ heavy(x) { return slow(x); } });

// main.ts
const api = Comlink.wrap(new Worker(...));
const r = await api.heavy(input);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое OffscreenCanvas?

02 Чем Comlink упрощает API?

03 Какой overhead на postMessage?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers

ВОПРОС Q21

Что нельзя делать в Web Worker?

ОПИСАНИЕ

Нельзя: обращаться к DOM, window, document, parent. Нельзя использовать alert, confirm. Нет доступа к большинству Web API напрямую (но можно fetch, WebSocket, IndexedDB, Cache API, Crypto). Нет общей памяти с main thread (кроме SharedArrayBuffer под cross-origin isolation).

КАК РАБОТАЕТ

- 01 Worker global – это WorkerGlobalScope (self), а не window.
- 02 Доступны: fetch, WebSocket, IndexedDB, Cache API, Crypto API, Notifications.
- 03 Недоступны: DOM, localStorage (есть IndexedDB), sessionStorage, alert.
- 04 SharedArrayBuffer + Atomics – общая память (требует COOP/COEP-headers).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Понимание для решения «можно ли вынести задачу в worker».
- 02 Code review – отлавливать `window.X` в worker-коде.
- 03 Архитектура: разделить логику UI и computation.

ПОДВОДНЫЕ КАМНИ

- 01 Попытка использовать DOM – ReferenceError.
- 02 localStorage недоступен – путаются.
- 03 Динамический import работает только в module worker.
- 04 Третьесторонние библиотеки часто используют window.

КОД

// Что доступно

```
// в worker.ts
fetch('/api');           // ok
indexedDB.open('db');    // ok
self.crypto.subtle.digest(...); // ok

document.title = 'x';    // ReferenceError
localStorage.setItem(...); // ReferenceError
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое SharedArrayBuffer?
- 02 Зачем COOP/COEP headers?
- 03 Как обойти отсутствие localStorage?

ПОЛНАЯ СТАТЬЯ

[https://developer.mozilla.org/en-US/docs/Web/API/Web Workers API/Using web workers#functions and classes available to workers](https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers#functions_and_classes_available_to_workers)

В О П Р О С Q 2 2

Как происходит communication с Worker?

ОПИСАНИЕ

Через message passing. Main: `worker.postMessage(data)`; слушает `worker.onmessage`. Worker: `self.postMessage(...)`; слушает `self.onmessage`. Данные сериализуются structured cloning algorithm (см. JS-fundamentals). Большие объекты – медленно; для них есть transferable objects (см. Q23). Удобные обёртки – Comlink (RPC-стиль), broadcast-channel.

КАК РАБОТАЕТ

- 01 `postMessage(data, [transferable])` – отправить.
- 02 `onmessage(event)` – получить, `event.data`.
- 03 Structured clone: копия всех структур (Map, Set, Date, ArrayBuffer).
- 04 Не сериализуются: функции, классы (теряют prototype), DOM-узлы.
- 05 Transferable – передача без копирования (см. Q23).
- 06 MessageChannel – двусторонний канал (port1/port2).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая работа с worker.
- 02 Между фреймами / iframe (через `postMessage`).
- 03 BroadcastChannel – между tabs.
- 04 Comlink – для RPC-style API.

ПОДВОДНЫЕ КАМНИ

- 01 Большие данные копируются – медленно.
- 02 Async-style сложен на `postMessage` – Comlink упрощает.
- 03 В разных версиях браузеров nuances structured clone.
- 04 Worker, оставленный без termination – память течёт.

КОД

// Двусторонний обмен

```
// main.ts
worker.postMessage({ id: 1, action: 'compute', payload });
worker.onmessage = (e) => {
  if (e.data.id === 1) doSomething(e.data.result);
};
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое structured cloning?
- 02 Чем MessageChannel помогает?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/Worker/postMessage>

ВОПРОС Q23

Что такое transferable objects?

ОПИСАНИЕ

Transferable – объекты, которые можно ПЕРЕДАТЬ между потоками вместо копирования. После передачи источник становится unusable (detached). Это ArrayBuffer, MessagePort, ImageBitmap, OffscreenCanvas, ReadableStream, WritableStream. Передача – мгновенная, без copying overhead.

КАК РАБОТАЕТ

- 01 `postMessage(data, [transferList])` – второй аргумент массив transferable.
- 02 После `postMessage` оригинал становится detached.
- 03 Чтение detached → ошибка.
- 04 Только specific types можно transfer.
- 05 Полезно для больших ArrayBuffer (audio/video/image).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Передача больших binary data в worker.
- 02 OffscreenCanvas → worker для рендера.
- 03 Audio/video processing.
- 04 ML-модели и weights.
- 05 Streams через worker boundary.

ПОДВОДНЫЕ КАМНИ

- 01 После transfer источник не работает – не пытаться читать.
- 02 Не все объекты transferable – только specific types.
- 03 Если хотите потом получить обратно – нужно transfer снова.

КОД

```
// Transfer ArrayBuffer
```

```
const buf = new ArrayBuffer(1024 * 1024);
worker.postMessage({ buf }, [buf]);
// buf теперь detached в main
buf.byteLength; // 0
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие объекты transferable?
- 02 Что такое detached state?
- 03 Чем OffscreenCanvas удобен в worker?

ПОЛНАЯ СТАТЬЯ

[https://developer.mozilla.org/en-US/docs/Web/API/Web Workers API/Transferable objects](https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Transferable_objects)

ДЕНЬ 6

НАГРУЗКА

ТЯЖЁЛЫЙ

4 ВОПРОСА

SERVICE WORKER**CACHE API****OFFLINE****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Сетевой прокси, кэширование, оффлайн-поведение.

ВОПРОСЫ ДНЯ

Q24 Что такое Service Worker?

Q25 Чем Service Worker отличается от Web Worker?

Q26 Что такое Cache API?

Q27 Как работает offline fallback?

Что такое Service Worker?

ОПИСАНИЕ

Service Worker – особый тип worker'а, который выступает как сетевой прокси между приложением и сетью. Перехватывает fetch-запросы, может отдавать ответ из кэша или собирать стратегии. Основа PWA: offline-режим, фоновые синхронизации, push-уведомления. Живёт независимо от вкладок, регистрируется один раз.

КАК РАБОТАЕТ

- 01 navigator.serviceWorker.register('/sw.js').
- 02 Lifecycle: install → activate → idle → terminated.
- 03 fetch event – перехват сетевого запроса.
- 04 Cache API – хранение ответов.
- 05 Background sync, Push, Notifications API.
- 06 Scope – определяет, какие URL контролирует.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 PWA – оффлайн поддержка.
- 02 Cache-стратегии (cache-first, network-first, stale-while-revalidate).
- 03 Push notifications.
- 04 Background sync для отправки данных при появлении сети.

ПОДВОДНЫЕ КАМНИ

- 01 Только HTTPS (кроме localhost).
- 02 Scope ограничен папкой регистрации.
- 03 Update lifecycle сложен – старая версия живёт, пока не закроют все вкладки.
- 04 Дебаг сложен – нужны DevTools Application tab.

КОД

// Регистрация

```
if ('serviceWorker' in navigator) {
  navigator.serviceWorker.register('/sw.js');
}

// sw.js
self.addEventListener('fetch', (e) => {
  e.respondWith(caches.match(e.request) || fetch(e.request));
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Какой lifecycle у Service Worker?
- 02 Что такое scope?
- 03 Зачем HTTPS обязателен?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

Чем Service Worker отличается от Web Worker?

ОПИСАНИЕ

Web Worker – общий поток для тяжёлых вычислений, живёт пока жива страница, общается с main через postMessage. Service Worker – специализированный сетевой прокси, живёт независимо от вкладок, перехватывает сетевые запросы, поддерживает Push. Web Worker – для CPU; Service Worker – для сети + оффлайн.

КАК РАБОТАЕТ

- 01 Web Worker: per-page, dedicated, message passing.
- 02 Service Worker: per-origin, общий для всех вкладок.
- 03 Web Worker не управляет сетью – только compute.
- 04 Service Worker имеет fetch event и Cache API.
- 05 Service Worker может быть terminated и пере-запущен.
- 06 Push/Notifications – только Service Worker.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 CPU задачи → Web Worker.
- 02 Offline / PWA → Service Worker.
- 03 Push-уведомления → Service Worker.
- 04 Background sync → Service Worker.

ПОДВОДНЫЕ КАМНИ

- 01 Думать, что Service Worker – это для всего, и пихать туда compute.
- 02 Web Worker нельзя использовать для сетевого перехвата.
- 03 Service Worker может быть terminated в любой момент – state не сохраняется.

КОД

// Назначение

```
// Web Worker – compute
const w = new Worker('compute.js');

// Service Worker – network
navigator.serviceWorker.register('/sw.js');
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Shared Worker?
- 02 Чем Push отличается от Notifications?
- 03 Что такое Workbox?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

Что такое Cache API?

ОПИСАНИЕ

Cache API – хранилище response'ов в Service Worker (и в обычной странице). Отличается от HTTP-cache: явно контролируется кодом, не подчиняется заголовкам. Используется в SW для оффлайн-режима. Структура: Cache → match/put/delete операции. Поддерживает несколько named-caches для версионирования.

КАК РАБОТАЕТ

- 01 caches.open('name') → Cache.
- 02 cache.put(request, response).
- 03 cache.match(request) → Response | undefined.
- 04 cache.delete / cache.keys.
- 05 caches.match(request) – поиск во всех кэшах.
- 06 Names используются для версионирования (v1, v2).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Оффлайн-кэширование статики (HTML/CSS/JS/fonts).
- 02 Cache-first стратегия для иконок и ассетов.
- 03 Stale-while-revalidate для API.
- 04 Pre-cache при install Service Worker.

ПОДВОДНЫЕ КАМНИ

- 01 Размер кэша ограничен (квота).
- 02 Старые кэши не удаляются автоматически – нужно в activate.
- 03 POST-запросы по умолчанию не кэшируются.
- 04 Заголовки HTTP-cache не учитываются в Cache API.

КОД

// Pre-cache

```
self.addEventListener('install', (e) => {
  e.waitUntil(caches.open('v1').then((c) =>
    c.addAll(['/ ', '/styles.css', '/app.js'])));
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем отличается Cache API от HTTP cache?
- 02 Как удалять старые кэши?
- 03 Что такое Workbox?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/Cache>

Как работает offline fallback?

ОПИСАНИЕ

Service Worker перехватывает fetch и решает, как ответить. Стратегии: cache-first (быстрее, но stale), network-first (свежее, fallback на cache), stale-while-revalidate (отдать кэш + обновить в фоне), cache-only (никогда сети), network-only (никогда кэша). При полном offline – fallback на cached offline.html.

КАК РАБОТАЕТ

- 01 fetch event → respondWith(...).
- 02 caches.match → может вернуть response.
- 03 fetch → если упал, fallback.
- 04 Generic offline page для любых неуспешных navigation.
- 05 Workbox упрощает написание стратегий.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 PWA с offline-режимом.
- 02 Mobile-приложения с нестабильной сетью.
- 03 Read-only приложения с локальной БД.
- 04 Документация / read-the-docs.

ПОДВОДНЫЕ КАМНИ

- 01 Network-first без timeout – медленно при плохой сети.
- 02 Cache-first – пользователь видит старое.
- 03 Без offline.html navigation падает.
- 04 Backend-side mutations не работают offline.

КОД

// Stale-while-revalidate

```
self.addEventListener('fetch', (e) => {
  e.respondWith(
    caches.match(e.request).then((cached) => {
      const fetchPromise = fetch(e.request).then((res) => {
        caches.open('v1').then((c) => c.put(e.request, res.clone()));
        return res;
      });
      return cached || fetchPromise;
    })
  );
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие 5 стратегий кэширования?
- 02 Что делать с POST-запросами оффлайн?
- 03 Что такое Background Sync?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/offline-cookbook>

PWA**UPDATES****DEBUG SW****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Progressive Web App: дизайн, обновления, отладка.

ВОПРОСЫ ДНЯ

Q28 Как проектировать PWA?

Q29 Как обновлять PWA без плохого UX?

Q30 Как отлаживать Service Worker?

ВОПРОС Q28

Как проектировать PWA?

ОПИСАНИЕ

PWA – приложение, которое можно установить, работает офлайн, поддерживает push. Чек-лист: HTTPS, manifest.json (icons, name, theme-color, display: standalone), Service Worker с offline-стратегией, Lighthouse PWA-audit ≥ 90 , responsive design, fast TTI. Архитектура: app-shell + dynamic content.

КАК РАБОТАЕТ

- 01 manifest.json – описание установленного приложения.
- 02 Service Worker – оффлайн.
- 03 Icons (192x192, 512x512) – для home-screen.
- 04 App-shell – кэш статичной оболочки, контент подгружается.
- 05 Push API – уведомления.
- 06 Web Share, Badging, Background Sync – дополнительные API.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Mobile-friendly приложения.
- 02 Сервисы, где offline критичен (заметки, tasklists).
- 03 Альтернатива нативным приложениям.
- 04 Долгоживущие SaaS-инструменты.

ПОДВОДНЫЕ КАМНИ

- 01 iOS / Safari – поддержка PWA ограничена.
- 02 App-store distribution требует TWA / PWABuilder.
- 03 Manifest без правильных icons → не устанавливается.
- 04 SW регистрируется на старте, но активируется не сразу.

КОД

// Manifest минимум

```
{
  "name": "My App",
  "short_name": "App",
  "start_url": "/",
  "display": "standalone",
  "theme_color": "#000",
  "background_color": "#fff",
  "icons": [{ "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" }]
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое app-shell architecture?
- 02 Какие limit'ы у iOS PWA?
- 03 Что такое Workbox?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/explore/progressive-web-apps>

ВОПРОС Q29

Как обновлять PWA без плохого UX?

ОПИСАНИЕ

Главная проблема – Service Worker «застревает» на старой версии, пока не закрыты все вкладки. Решения: при обнаружении новой SW показать ненавязчивый toast «доступно обновление», при клике вызвать `skipWaiting + reload`. Workbox имеет готовые helpers. Альтернатива – `auto-skipWaiting` (быстрее, но обновление в моменте может быть неожиданным).

КАК РАБОТАЕТ

- 01 `registration.update()` – проверить новую версию.
- 02 При новой версии SW переходит в `waiting` state.
- 03 `self.skipWaiting()` в новой SW – активировать сразу.
- 04 `clients.claim()` – взять control над открытыми вкладками.
- 05 Toast/banner: «обновление доступно, нажмите для перезагрузки».
- 06 Workbox `messageSkipWaiting()` – стандартный приём.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любое PWA в production.
- 02 После каждого деплоя.
- 03 При hot-fix критичных багов.

ПОДВОДНЫЕ КАМНИ

- 01 Auto-skip в момент пользователя печатает форму = data loss.
- 02 Без update flow пользователь годами на старой версии.
- 03 Cache не invalidate-ится → новый код не загружается.
- 04 iOS обновляет SW по другим правилам.

КОД

// Update toast

```
navigator.serviceWorker.register('/sw.js').then((reg) => {
  reg.addEventListener('updatefound', () => {
    const newSW = reg.installing!;
    newSW.addEventListener('statechange', () => {
      if (newSW.state === 'installed' && navigator.serviceWorker.controller) {
        showUpdateBanner(() => {
          newSW.postMessage({ type: 'SKIP_WAITING' });
          window.location.reload();
        });
      }
    });
  });
});
```

```
});  
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Что делает skipWaiting?
- 02 Чем clients.claim() помогает?
- 03 Как протестировать update flow?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/service-worker-lifecycle>

ВОПРОС Q30

Как отлаживать Service Worker?

ОПИСАНИЕ

Chrome DevTools → Application → Service Workers: видны зарегистрированные SW, их state, можно unregister, обновить, симулировать offline. Application → Cache Storage – содержимое кэшей. Network → Disable cache + Offline checkbox. Console внутри SW – отдельный context. Ошибки логируются в Application → Service Workers.

КАК РАБОТАЕТ

- 01 DevTools → Application → Service Workers.
- 02 Update on reload – auto-update при F5.
- 03 Unregister, Stop, Skip waiting – кнопки.
- 04 Network → Offline/Slow 3G – симуляция.
- 05 Console внутри SW context.
- 06 chrome://serviceworker-internals – детальный список.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 При разработке любого SW.
- 02 При баг-репортах от пользователей.
- 03 При тестировании cache-стратегий.
- 04 При QA offline-режима.

ПОДВОДНЫЕ КАМНИ

- 01 Ленивый dev-цикл – забыли «Update on reload», видят старую версию.
- 02 Старые SW застряли – нужно unregister.
- 03 Cache хранится между релизами – не очищается автоматически.
- 04 Sentry / Rollbar в SW требуют особой настройки.

КОД

// DevTools shortcuts

```
// Application → Service Workers
// ✓ Update on reload – рекомендуется в dev
// 'Bypass for network' – отладка без SW
// 'Offline' – симуляция оффлайна
```

МОГУТ ДОСПРОСИТЬ

- 01 Что показывает chrome://serviceworker-internals?
- 02 Как тестировать SW в CI?
- 03 Чем Workbox упрощает дебаг?

ПОЛНАЯ СТАТЬЯ

<https://developer.chrome.com/docs/devtools/progressive-web-apps>

ФИНАЛЬНЫЙ ЧЕК - ЛИСТ

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ [] Описать IntersectionObserver, ResizeObserver, MutationObserver и риски (Q1-Q5)
- ☐ [] Объяснить event delegation и passive listeners (Q6-Q10)
- ☐ [] Расписать capture/bubble/stopPropagation/preventDefault (Q11-Q13)
- ☐ [] Описать composed events и focus management (Q14-Q18)
- ☐ [] Объяснить, что такое Web Worker, что нельзя, communication, transferable (Q19-Q23)
- ☐ [] Описать Service Worker, Cache API, offline стратегии (Q24-Q27)
- ☐ [] Описать дизайн PWA, обновления и отладку Service Worker (Q28, Q29, Q30)

EVENT LOOP / ASYNC МЕТОДИЧКА v1

МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru